

作业 04

1、阅读下列 C 程序，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【C程序】

```
Int DoString(char*string){
    char *argv[100];
    Int  argc=1;
    while(1) {                                //1
        while(*string&& *string!='-')          //2,3
            String++;                          //4
        if(!*string)                          //5
            break;                             //6
        argv[argc]=string;
        while(*string && *string!="&& *string!='\n'&& *string!= '\t') //7,8,9,10
            string++;                          //11
        argc++;                               //12
    }
    return 0;                                //13
}
```

【问题 1】

请针对上述 C 程序给出满足 100%DC（判定覆盖）所需的逻辑条件。

答案：

判定覆盖（DC）要求每个判定（*decision*）的真假分支都至少执行一次。程序中
共有 4 个判定节点。

编号	判定语句	所在行	真分支条件	假分支条件
1	while(1)	1	恒为真，总是进入循	恒为假，不可达（但

编号	判定语句	所在行	真分支条件	假分支条件
			环体	结构上存在该分支)
2	<code>while(*string && *string!='-')</code>	2-3	<code>*string != '\0'</code> 且 <code>*string != '-'</code> , 例如 <code>string = "abc"</code> (首个字符为 'a')	<code>*string == '\0'</code> 或 <code>*string == '-'</code> , 例如 <code>string = ""</code> (空串) 或 <code>string = "-x"</code> (首字符为 '-')
3	<code>if(!*string)</code>	5	<code>*string == '\0'</code> , 即字符串已到结尾, 例如 <code>string</code> 指向 '\0'	<code>*string != '\0'</code> , 即字符串未到结尾, 例如 <code>string</code> 指向 '-'
4	<code>while(*string && *string!=' ' && *string!='\n' && *string!='\t')</code>	7-10	<code>*string != '\0'</code> 且不为空格、换行、制表符, 例如 <code>string = "-abc"</code> ('-' 后是普通字符)	<code>*string == '\0'</code> 或为空格/换行/制表符, 例如 <code>string = "- "</code> ('-' 后是空格)

推荐测试用例：

用例编号	输入 <i>string</i>	覆盖的判定分支	说明
TC1	"" (空串)	判定 2-假, 判定 3-真, <i>break</i>	字符串为空, 直接退出
TC2	"-a"	判定 2-假, 判定 3-假, 判定 4-真→假	含一个短选项, 覆盖正常解析流程
TC3	"abc"	判定 2-真→假, 判定 3-真, <i>break</i>	无 '-' 的字符串, 跳过字符后结束
TC4	"-a -b"	判定 2-假, 判定 3-假, 判定 4-真→假 →判定 2-假, 判定 3-假, 判定 4-真→假	多个选项, 覆盖循环

【问题 2】

请画出上述程序的控制流图，并计算其控制流图的环路复杂度 $V(G)$ 。

答案：

(1) 控制流图：

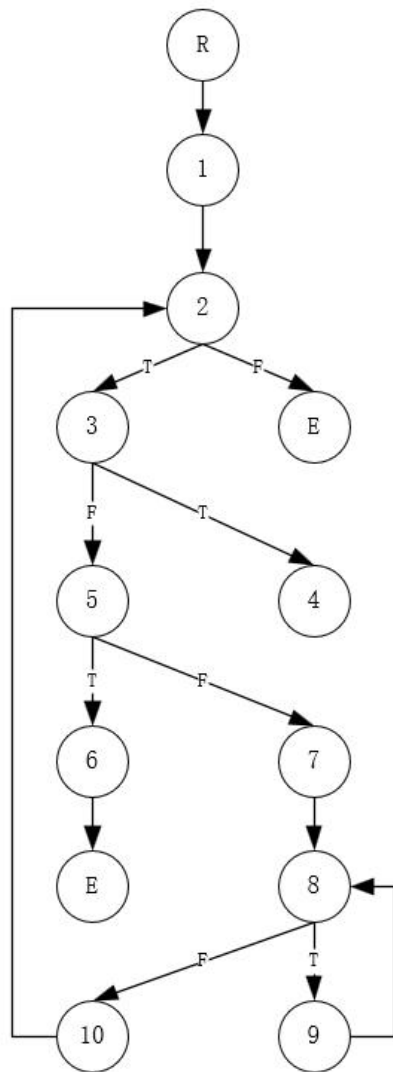
将程序划分为以下节点：

节点	对应代码	说明
<i>R</i>	—	入口 (<i>Start</i>)
1	char *argv[100]; int argc=1;	初始化
2	while(1)	外层循环判定
3	while(*string && *string!='-')	第一内层循环判定
4	string++;	第一内层循环体
5	if(!*string)	<i>if</i> 判定
6	break;	跳出外层循环
7	argv[argc]=string;	记录参数位置
8	while(*string && *string!=' ' && *string!='\n' && *string!='\t')	第二内层循环判定
9	string++;	第二内层循环体
10	argc++;	参数计数加一
<i>E</i>	return 0;	出口 (<i>End</i>)

控制流图（有向边集合）：

R → 1
1 → 2
2 → 3 (while(1) 真分支)
2 → *E* (while(1) 假分支，理论存在但不可达)
3 → 4 (第一内层 while 真分支：*string 非空且非'-')
3 → 5 (第一内层 while 假分支：*string 为空或为'-')
4 → 3 (循环回边)

5 → 6 (if 真分支: *string 为空)
 5 → 7 (if 假分支: *string 非空, 即遇到 '-')
 6 → E (break 到出口)
 7 → 8 (顺序执行)
 8 → 9 (第二内层 while 真分支: *string 非空且非空白字符)
 8 → 10 (第二内层 while 假分支: *string 为空或为空白字符)
 9 → 8 (循环回边)
 10 → 2 (循环回边到外层 while)



(2) 环路复杂度 $V(G)$:

方法一 (判定节点法): $V(G) = P + 1$

判定节点有 4 个：节点 2 (*while*(1))、节点 3（第一个内层 *while*）、节点 5 (*if*)、节点 8（第二个内层 *while*）。

$V(G) = 4 + 1 = 5$

方法二（边-节点法）： $V(G) = E - N + 2$

边数 $E = 15$ ($R \rightarrow 1, 1 \rightarrow 2, 2 \rightarrow 3, 2 \rightarrow E, 3 \rightarrow 4, 3 \rightarrow 5, 4 \rightarrow 3, 5 \rightarrow 6, 5 \rightarrow 7, 6 \rightarrow E, 7 \rightarrow 8, 8 \rightarrow 9, 8 \rightarrow 10, 9 \rightarrow 8, 10 \rightarrow 2$)

节点数 $N = 12$ ($R, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, E$)

$V(G) = 15 - 12 + 2 = 5 \checkmark$

【问题 3】

请给出问题 2 中控制流图的线性无关路径。

答案：

$V(G) = 5$ ，因此有 5 条线性无关路径：

路径编号	路径序列	路径说明	覆盖场景
Path 1	$R \rightarrow 1 \rightarrow 2 \rightarrow E$	<i>while</i> (1) 假分支直接退出	理论路径 (<i>while</i> (1) 实际不可达假分支)
Path 2	$R \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow E$	字符串为空：第一内层 <i>while</i> 假 $\rightarrow$ <i>if</i> 真 $\rightarrow$ <i>break</i>	输入为空串 ""
Path 3	$R \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow E$	字符串有普通字符后结束：第一内层 <i>while</i> 真（执行循环体） $\rightarrow$ 假 $\rightarrow$ <i>if</i> 真 $\rightarrow$ <i>break</i>	输入为无 '-' 的字符串如 "abc"
Path 4	$R \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 7 \rightarrow$	字符串以 '-' 开头且后跟空白：第一内层 <i>while</i> 假 $\rightarrow$ <i>if</i> 假 $\rightarrow$	输入以 '-' 开头且紧随空白，如 "- "

路径编号	路径序列	路径说明	覆盖场景
	8 → 10 → 2 → E	第二内层 while 假 → argc++ → 回到外循环	或 "-\t"
Path 5	R → 1 → 2 → 3 → 5 → 7 → 8 → 9 → 8 → 10 → 2 → E	完整执行路径：第一内层 while 假 → if 假 → 第二内层 while 真（进入循环体）→ 假 → argc++ → 回到外循环	输入为有效选项如 "-a"

说明： 路径 Path 1 (while(1) 假分支) 在程序实际运行中不可达，但在控制流图结构中存在该边，因此作为一条线性无关路径。实际测试中可以忽略此路径，使用其余 4 条路径覆盖所有可达分支。

2、阅读下列 C 程序，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【C 程序】

```
static void permute_args(int panonopt_start, int panonopt_end, int opt_end, int ncycle)
    int cstart, cyclelen, i, j, nnonopts, nopts, pos; //1

    nnonopts = panonopt_end - panonopt_start;
    nopts = opt_end - panonopt_end;
    cyclelen = (opt_end - panonopt_start)/ ncycle;

    for(i=0; i < ncycle; i++){ //2
        cstart = panonopt_end+i; //3
        pos = cstart;
        for(j=0; j< cyclelen; j++){ //4
            if (pos >= panonopt_end) //5
                pos -= nnonopts; //6
            else //7
                pos += nopts;

        }
    } //8
```

【问题 1】

请针对上述 C 程序给出满足 100%DC（判定覆盖）所需的逻辑条件。

答案：

判定覆盖（DC）要求每个判定的真假分支都至少执行一次。程序中共有 3 个判定节点。

编号	判定语句	所在行	真分支条件	假分支条件
1	for(i=0; i < ncycle; i++) (即 i < ncycle)	2	i < ncycle 成立, 即 $ncycle > 0$ 时循环体会执行。例如 $ncycle = 3$	i < ncycle 不成立, 即 $ncycle \leq 0$ 或循环结束后。例如 $ncycle = 0$
2	for(j=0; j < cyclelen; j++) (即 j < cyclelen)	4	j < cyclelen 成立, 即 $cyclelen > 0$ 时循环体会执行。例如 $cyclelen = 2$	j < cyclelen 不成立, 即 $cyclelen \leq 0$ 或循环结束后。例如 $cyclelen = 0$
3	if (pos >= panonopt_end)	5	pos >= panonopt_end 成立, 执行 pos -= nnonopts	pos < panonopt_end 成立, 执行 pos += nopts

推荐测试用例（参数组合）：

设 $panonopt_start = 0$ ，构造以下测试用例：

用例编号	输入参数	覆盖的判定分支	说明
TC1	$panonopt_start=0$, $panonopt_end=2$, $opt_end=4$, $ncycle=0$	判定 1-假	$ncycle=0$, 外循环不执行
TC2	$panonopt_start=0$, $panonopt_end=2$, $opt_end=2$, $ncycle=1$	判定 1-真 →假, 判定 2-假	$cyclelen=0$, 内循环不执行
TC3	$panonopt_start=0$, $panonopt_end=2$,	判定 1-真 →假, 判定	$cyclelen=3$, 内循环执行, $pos \geq$

用例编号	输入参数	覆盖的判定分支	说明
	<i>opt_end=5, ncycle=1</i>	2-真→假, 判定 3-真	<i>panonopt_end</i> 为真
TC4	<i>panonopt_start=0,</i> <i>panonopt_end=4,</i> <i>opt_end=6, ncycle=1</i>	判定 1-真 →假, 判定 2-真→假, 判定 3-假	初始 <i>pos</i> < <i>panonopt_end</i> 时, <i>pos</i> ≥ <i>panonopt_end</i> 为假

【问题 2】

请画出上述程序的控制流图，并计算其控制流图的环路复杂度 $V(G)$ 。

答案：

(1) 控制流图：

将程序划分为以下节点：

节点	对应代码	说明
R	—	入口 (<i>Start</i>)
1	变量声明 + <i>nnonopts/nopts/cyclelen</i> 赋值	初始化
2	<i>for(i=0; i < ncycle; i++)</i> 判定	外循环判定 (<i>i < ncycle</i>)
3	<i>cstart = panonopt_end+i; pos = cstart;</i>	外循环体开始
4	<i>for(j=0; j < cyclelen; j++)</i> 判定	内循环判定 (<i>j < cyclelen</i>)
5	<i>if(pos >= panonopt_end)</i> 判定	<i>if</i> 判定
6	<i>pos -= nnonopts;</i>	<i>if</i> 真分支
7	<i>pos += nopts;</i>	<i>if</i> 假分支
8	<i>j++</i> (内循环增量, 回到判定 4)	内循环迭代

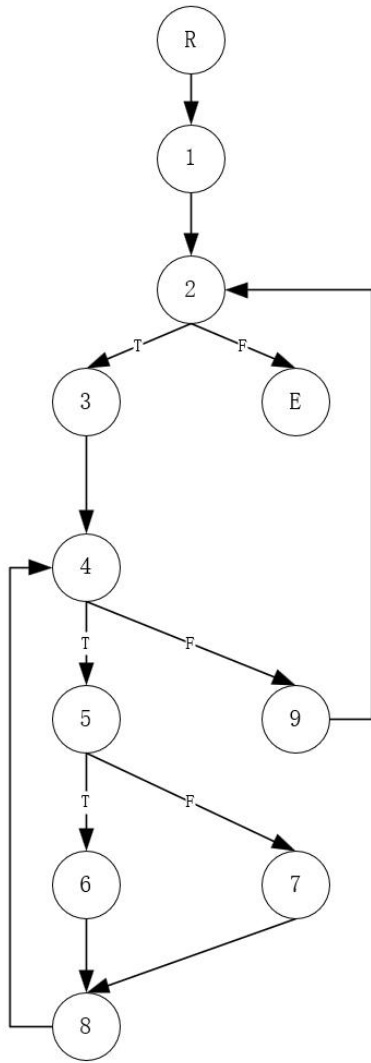
节点	对应代码	说明
9	i++ (外循环增量, 回到判定 2)	外循环迭代
E	—	出口 (End)

控制流图 (有向边集合) :

```

R  → 1
1  → 2
2  → 3    (外循环真分支: i < ncycle)
2  → E    (外循环假分支: i >= ncycle)
3  → 4    (顺序执行)
4  → 5    (内循环真分支: j < cyclelen)
4  → 9    (内循环假分支: j >= cyclelen)
5  → 6    (if 真分支: pos >= panonopt_end)
5  → 7    (if 假分支: pos < panonopt_end)
6  → 8    (顺序执行)
7  → 8    (顺序执行)
8  → 4    (循环回边: j++ 后回到内循环判定)
9  → 2    (循环回边: i++ 后回到外循环判定)

```



(2) 环路复杂度 $V(G)$:

方法一（判定节点法）： $V(G) = P + 1$

判定节点有 3 个：节点 2（外循环判定 $i < ncycle$ ）、节点 4（内循环判定 $j < cyclelen$ ）、节点 5（if 判定 $pos \geq panonopt_end$ ）。

$$V(G) = 3 + 1 = 4$$

方法二（边-节点法）： $V(G) = E - N + 2$

边数 $E = 13$ ($R \rightarrow 1$, $1 \rightarrow 2$, $2 \rightarrow 3$, $2 \rightarrow E$, $3 \rightarrow 4$, $4 \rightarrow 5$, $4 \rightarrow 9$, $5 \rightarrow 6$, $5 \rightarrow 7$, $6 \rightarrow 8$, $7 \rightarrow 8$, $8 \rightarrow 4$, $9 \rightarrow 2$)

节点数 $N = 11$ ($R, 1, 2, 3, 4, 5, 6, 7, 8, 9, E$)

$V(G) = 13 - 11 + 2 = 4 \checkmark$

【问题 3】

请给出问题 2 中控制流图的线性无关路径。

答案：

$V(G) = 4$ ，因此有 4 条线性无关路径：

路径编号	路径序列	路径说明	覆盖场景
Path 1	$R \rightarrow 1 \rightarrow 2 \rightarrow E$	外循环不执行 ($i < ncycle$ 为假)	$ncycle \leq 0$ ，函数直接返回
Path 2	$R \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 9 \rightarrow 2 \rightarrow E$	外循环执行但内循环不执行 ($j < cyclelen$ 为假)	$ncycle > 0$ 但 $cyclelen \leq 0$
Path 3	$R \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 8 \rightarrow 4 \rightarrow 9 \rightarrow 2 \rightarrow E$	内循环执行且 if 为真 ($pos \geq panonopt_end$)	$cyclelen > 0$ ，且当前 pos 在非选项区域
Path 4	$R \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 7 \rightarrow 8 \rightarrow 4 \rightarrow 9 \rightarrow 2 \rightarrow E$	内循环执行且 if 为假 ($pos < panonopt_end$)	$cyclelen > 0$ ，且当前 pos 在选项区域

路径覆盖说明：

- Path 1 覆盖判定 1 的假分支（外循环不执行）
- Path 2 覆盖判定 1 的真分支 + 判定 2 的假分支（外循环执行，内循环不执行）

- *Path 3* 覆盖判定 2 的真分支 + 判定 3 的真分支（内循环执行，执行 `pos -= nnonopts`）
- *Path 4* 覆盖判定 2 的真分支 + 判定 3 的假分支（内循环执行，执行 `pos += nopts`）

这 4 条路径覆盖了控制流图中所有有向边，构成一组基路径。